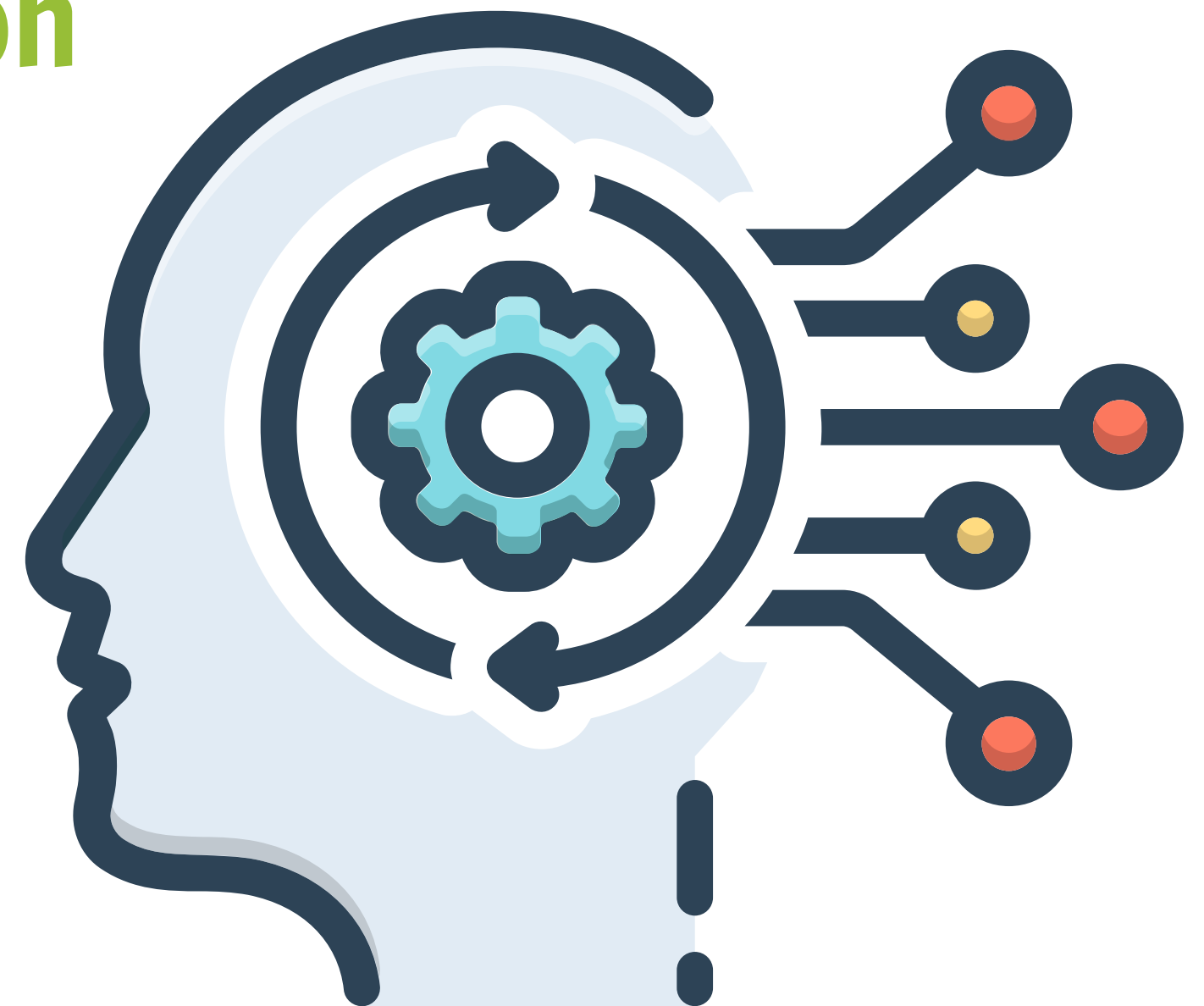


Conteneurisation & orchestration

Docker Compose

INFO – AGROTIC

Last update: oct. 2025



Docker Compose - Introduction

Qu'est-ce que Docker Compose ?

Docker Compose est un outil pour définir et gérer des applications multi-conteneurs.

Avantages :

- Configuration déclarative en YAML
- Démarrage de tous les services en une commande
- Gestion des réseaux et volumes automatique
- Idéal pour le développement et les tests

Docker Compose - Introduction

Commandes principales :

docker-compose up # Démarrer les services

docker-compose down # Arrêter et supprimer

docker-compose ps # Lister les conteneurs

docker-compose logs # Voir les logs

Structure d'un fichier docker-compose.yml

**Le fichier docker-compose.yml
définit l'architecture de votre
application.**

services:
Définit les conteneurs

```
version: '3.8'

services:
  web:
    image: nginx:latest
    ports: ["8080:80"]
    volumes: [".:/html:/usr/share/nginx/html"]
    networks: ["frontend"]

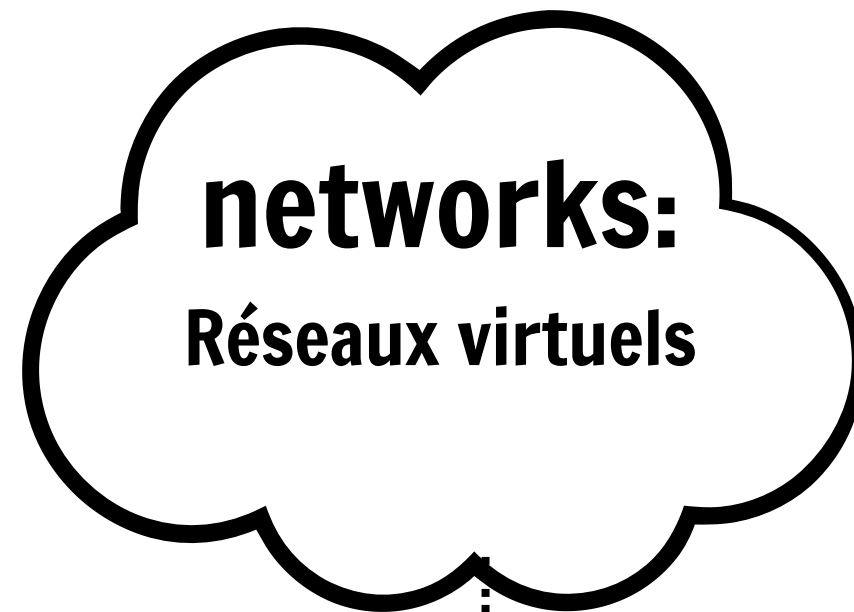
  database:
    image: postgres:15
    environment:
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: mydb
    volumes: ["db-data:/var/lib/postgresql/data"]
    networks: ["backend"]

networks:
  frontend:
  backend:

volumes:
  db-data:
```

Structure d'un fichier docker-compose.yml

Le fichier docker-compose.yml définit l'architecture de votre application.



```
version: '3.8'
services:
  web:
    image: nginx:latest
    ports: ["8080:80"]
    volumes: ["/usr/share/nginx/html"]
    networks: ["frontend"]

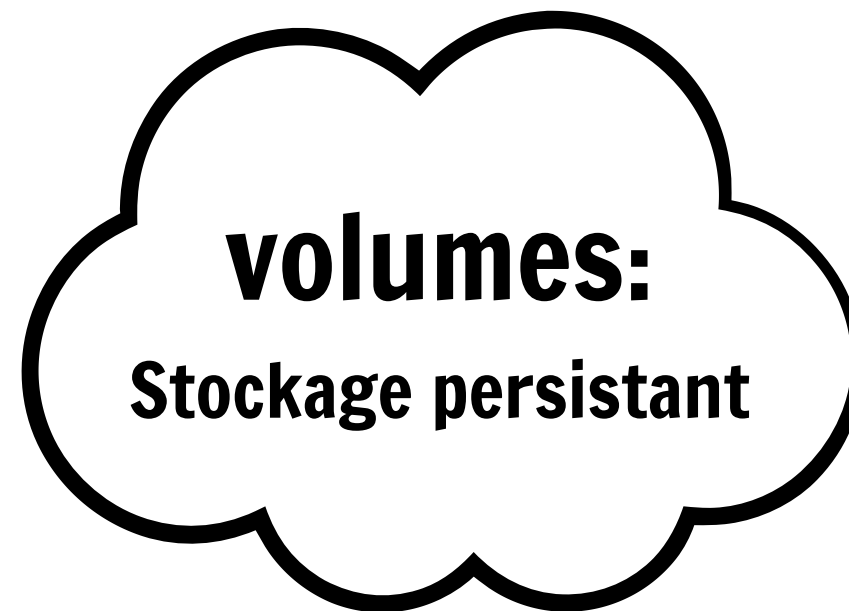
  database:
    image: postgres:15
    environment:
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: mydb
    volumes: ["db-data:/var/lib/postgresql/data"]
    networks: ["backend"]

networks:
  frontend:
  backend:

volumes:
  db-data:
```


Structure d'un fichier docker-compose.yml

Le fichier docker-compose.yml définit l'architecture de votre application.



```
version: '3.8'
services:
  web:
    image: nginx:latest
    ports: ["8080:80"]
    volumes: ["/usr/share/nginx/html"]
    networks: ["frontend"]

  database:
    image: postgres:15
    environment:
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: mydb
    volumes: ["db-data:/var/lib/postgresql/data"]
    networks: ["backend"]

networks:
  frontend:
  backend:

volumes:
  db-data:
```

Exemple 1 – Application Web (NGINX) + BD PostgreSQL

Fonctionnement

- **web** sert du contenu HTML local via NGINX
- **database** utilise PostgreSQL avec stockage persistant
- Les **deux services** sont sur le même réseau pour pouvoir communiquer

```
version: '3.9'

services:
  web:
    image: nginx:latest
    ports: ["8080:80"]
    volumes: [".:/html:/usr/share/nginx/html"]
    depends_on: ["database"]
    networks: ["stack"]

  database:
    image: postgres:16
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: password
      POSTGRES_DB: mydb
    volumes: ["db:/var/lib/postgresql/data"]
    networks: ["stack"]

networks:
  stack:

volumes:
  db:
```


Exemple Docker Compose – Django + PostgreSQL

Dockerfile

```
FROM python:3.12
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
```

 requirements.txt

php

django

psycopg2-binary

```
version: "3.9"
```

```
services:
```

```
  web:
```

```
    build: ./django
```

```
    command: python manage.py runserver 0.0.0.0:8000
```

```
    ports: ["8000:8000"]
```

```
    volumes: [".:/django:/app"]
```

```
    depends_on: ["db"]
```

```
    networks: ["django-net"]
```

```
  db:
```

```
    image: postgres:15
```

```
    environment:
```

```
      POSTGRES_DB: mydb
```

```
      POSTGRES_USER: user
```

```
      POSTGRES_PASSWORD: password
```

```
    volumes: ["pgdata:/var/lib/postgresql/data"]
```

```
    networks: ["django-net"]
```

```
networks:
```

```
  django-net:
```

```
volumes:
```

```
  pgdata:
```


Exercices d'application (1)

Exercice 1 — Déployer NGINX et personnaliser le contenu

Objectif

Déployer un serveur web NGINX avec Docker Compose et afficher une page HTML personnalisée.

Instructions

1. Crée un dossier `ex1`
2. À l'intérieur, crée un dossier `html` contenant un fichier `index.html`
3. Écris un fichier `docker-compose.yml` qui :
 - Lance un service NGINX
 - Monte ton HTML dans `/usr/share/nginx/html`
 - Expose le site sur ton navigateur via `localhost:8080`

Bonus

- Modifier la page HTML pendant que le conteneur tourne et observer le résultat.
- Ajouter un deuxième service `alpine` en mode test pour accéder au réseau.

Exercices d'application (2)

Exercice 2 — Node.js API + MongoDB

Objectif

Composer deux services qui communiquent sur un réseau Docker.

Instructions

1. Crée un dossier `api` avec une petite API Node.js (ex: `hello world`)
2. Crée un `Dockerfile` dans `/api`
3. Dans `docker-compose.yml` :
 - Service `api` basé sur ton Dockerfile
 - Service `mongo` basé sur l'image officielle `mongo:6`
 - Variable d'environnement indiquant `MONGO_URL=mongodb://mongo:27017/dbtest`
4. Lancer avec `docker compose up -d`
5. Tester `http://localhost:3000`

Bonus

- Stocker un compteur dans MongoDB et afficher sa valeur.
- Ajouter un volume persistant pour MongoDB.

Exercices d'application (3)

Exercice 3 — Laravel + MySQL + phpMyAdmin

Objectif

Déployer un environnement de développement web complet avec Docker Compose.

Instructions

1. Crée un dossier `ex3`
2. Installer un projet Laravel (`laravel new` ou `composer create-project`)
3. Ajouter :
 - Service `app` basé sur un Dockerfile
 - Service `mysql` avec variables `MYSQL_ROOT_PASSWORD`, `MYSQL_DATABASE=laravel`
 - Service `phpmyadmin` accessible sur `localhost:8081`
4. Vérifier que Laravel se connecte à MySQL via le network interne (`DB_HOST=mysql`)
5. Test : créer une table via migrations

Bonus

- Ajouter un volume nommé pour MySQL
- Ajouter `depends_on` pour contrôler l'ordre de démarrage

